

Using Git and GitHub in This Course

Git and GitHub are part of the standard course workflow. You will usually use them to get course code, track your work, save progress, and submit assignments in a reliable way.

What Git and GitHub are

Git and GitHub are related, but they are not the same thing.

- Git is a local version control tool. It tracks changes on your computer inside a repository.
- GitHub is an online platform for hosting, sharing, and submitting Git repositories.

A simple way to think about it is:

- Git helps you manage your work locally.
- GitHub helps you store and share that work online.

Why use Git and GitHub

Students usually care about a few practical benefits:

- Saving progress safely: commits give you restore points while you work.
- Tracking changes: you can see what changed and when it changed.
- Recovering from mistakes: Git makes it easier to go back, compare versions, and fix problems.
- Submitting work reliably: GitHub gives the course a clear place to find your code.
- Collaborating or sharing when needed: you can share your repository with the instructor or teammates if the course uses that workflow.

How Git and GitHub fit into this course

In this course, the default Git workflow is simple.

Most of the time, you will:

1. Clone a course repository from GitHub.
2. Make changes locally on your computer.
3. Check `git status`.
4. Commit meaningful work.

5. Pull if needed.
6. Push your work to GitHub.

You do not need to be an advanced Git user to succeed in the course. The most important habit is checking your status often and committing work in small, understandable steps.

Getting started

These are the first Git commands most students need:

```
git --version
git config --global user.name "Your Name"
git config --global user.email "your-email@example.edu"
git clone <repository-url>
git status
```

What these commands do:

- `git --version` checks that Git is installed.
- `git config --global user.name "Your Name"` sets the name attached to your commits.
- `git config --global user.email "your-email@example.edu"` sets the email attached to your commits.
- `git clone <repository-url>` downloads a repository to your computer.
- `git status` shows what Git sees in your current repository.

If Git is not installed yet, use the course setup guide before continuing.

Common commands

In this course, `git status`, `git add`, `git commit`, `git pull`, and `git push` are the commands you will use most often.

Command	What it does	Typical use in this course
<code>git --version</code>	Shows the installed Git version	Quick check that Git is available
<code>git config --global --list</code>	Shows your Git configuration	Check your name and email
<code>git clone <repository-url></code>	Downloads a repository	Standard first step for a new class repo

<code>git status</code>	Shows changed, staged, and untracked files	Use often while working
<code>git add <file></code>	Stages a file for commit	Prepare selected work for a commit
<code>git commit -m "message"</code>	Saves a snapshot with a message	Record meaningful progress
<code>git pull</code>	Gets new changes from GitHub	Update before pushing if needed
<code>git push</code>	Sends your commits to GitHub	Submit or back up your work
<code>git log --oneline</code>	Shows a short commit history	Review recent commits

Examples:

```
git status
git add notebook.ipynb
git commit -m "Finish setup check notebook"
git pull
git push
git log --oneline
```

Typical course workflows

Cloning a class repository

1. Copy the repository URL from GitHub or the course page.
2. Open a terminal where you want the folder to live.
3. Run `git clone <repository-url>`.
4. Move into the new repository folder.
5. Run `git status` to confirm Git recognizes the repository.

Checking what changed

1. Open a terminal in the repository folder.
2. Run `git status`.
3. Read which files are modified, staged, or untracked.
4. Decide what should be committed now and what should wait.

Committing a finished piece of work

1. Run `git status`.
2. Stage the file or files you want to save with `git add`.
3. Run `git commit -m "your message"`.
4. Run `git status` again to confirm the commit worked.

Example:

```
git add src/setup_check.py
git commit -m "Complete setup check script"
```

Pulling remote changes safely

1. Save your local work first.
2. Run `git status` and make sure you understand what is uncommitted.
3. Run `git pull`.
4. If Git reports a conflict, stop and read the message carefully before making more changes.

Pushing work to GitHub

1. Make sure you have committed your work locally.
2. Run `git push`.
3. Check GitHub in the browser if you want to confirm the new commit appears online.

Common problems and fixes

You are not in the repository folder

Symptoms:

- Git says not a `git repository`
- commands that should work in class repositories fail immediately

What to try:

- Make sure you opened a terminal inside the correct project folder.
- Use your terminal to move into the repository directory.
- Run `git status` again.

Authentication failure

Symptoms:

- Git asks for credentials repeatedly

- push or clone fails with an authentication error

What to try:

- Check that you are signed into the correct GitHub account.
- Use the course-approved authentication method.
- If the course uses organization access, verify that you accepted any invite.
- Ask for help if authentication still fails after checking your account.

Nothing was added before commit

Symptoms:

- Git says there is nothing to commit
- you expected a file to be saved, but it was not included

What to try:

- Run `git status`.
- Look for files listed as modified or untracked.
- Stage the right files with `git add <file>`.
- Run the commit again.

Push rejected because the remote changed

Symptoms:

- Git says your push was rejected
- Git says the remote contains work that you do not have locally

What to try:

- Run `git pull`.
- Read any messages carefully.
- If the pull succeeds, try `git push` again.
- If there is a conflict, stop and ask for help if you are unsure how to resolve it.

Merge conflict anxiety

Symptoms:

- Git mentions a merge conflict
- one or more files now contain conflict markers

What to do first:

- Do not panic.
- Do not start randomly deleting lines.
- Read the Git message and identify which file has the conflict.

- Save your work and ask for help if you are not sure what each version means.

Committing files that should not be tracked

Symptoms:

- `git status` shows files you do not want to submit
- large outputs, temporary files, or environment files appear in the repository

What to try:

- Check the repository `.gitignore`.
- Avoid adding files unless you know they belong in the repository.
- If you are unsure whether something should be committed, ask before pushing.

When to ask for help

Ask for help early if Git starts feeling confusing. When you ask, include:

- operating system
- the exact Git command you ran
- the full error message as text
- the current `git status` output
- what you already tried

That information makes it much easier for someone to help you quickly.